DA3 Review — Short Script (deck, project, demo, site)

Learned Falsehoods Look Like Knowledge · Goutham Krishna Jayakrishnan (24BCE1910), Shubham Khadilkar (24BCE5275)

About 15 minutes: slides 1–12 (9 min) → live demo (5 min) → slides 14–16 and questions.

0. Ten minutes before

1. Close Chrome and every other heavy program (the GPU has only 8 GB).
2. Open VS Code: `code C:\Stuff\Hallucination`. Open a terminal with **Ctrl+`** (the venv is already active).
3. Warm-up run, so nothing surprises you later: `python scripts\live_run.py --n 20` (about 20 s, should say 100.0 % and 20/20).
4. Open `presentation\DA3_Hallucination_Detection.pptx` and start the slide show from slide 1.

1. The project in four sentences (say this if asked "what did you do?")

"An AI model gives a probability to every word it writes, for free. We built a system that reads these probabilities from a single answer and flags answers that are likely wrong. We tested it fairly: 3 models, 3 datasets, 1,417 new questions, predictions written down before the test. It catches made-up facts and science mistakes, beats methods that need six calls, and we discovered its blind spot: misconceptions the model learned look exactly like knowledge."

2. Slides (what to say, in short)

Slide	Say
1 Title	"We built a cheap way to catch an AI when it makes things up, by reading how confident it is in each word. Today: the implementation, results on 1,400+ new questions, and our main discovery."
2 Free confidence score	"Every word the model writes has a probability. If the model is unsure, these drop, so we flag the answer. One call, instead of asking the model six times."
3 Stress-testing DA1	"First we checked our own DA1 work. We found a hidden repetition penalty that distorted the numbers, replaced word-matching grading with a rubric-based AI grader, and grew the test from 60 to 1,417 questions."
4 How the system works	"The model gives its most likely answer. We feed it back and record the confidence of every word. Check: every word written is the word the model rated most likely, so the numbers are really the model's. Then 17 measures, 10 detectors, compared with the grader's labels."

5 A fair test	"3 small open models on our laptop GPU; science, trivia and TruthfulQA, which is built to trap models with misconceptions. Designed on 200 practice questions, then one final test on 1,417 new ones."
6 Result 1	"0.5 is guessing, 1.0 is perfect. Science about 0.7 for all three models, trivia up to 0.81. Weaker on misconception questions; explaining why is our discovery."
7 Result 2	"Our one-call detector beat every multi-call method in 6 of 6 settings, at one sixth of the cost."
8 Result 3	"Made-up facts are caught: the model is less sure of them. But when it repeats a misconception, like 'SOS = Save Our Souls', it is as confident as when it is right. Other methods share this blind spot."
9 Confident mistakes	"Real examples. On 11 questions all three models give the same misconception, confidently. SmolLM2's wrong Armstrong quote is more confident than 99.7 % of its answers."
10 Tried to break it	"We matched answer length, controlled difficulty and topic, used a second independent grader, changed the prompt, tried a bigger model. For Qwen and SmolLM2 it held in almost every check; for Llama it depends on the grader, and we say so."
11 Difficulty	"Where it works, confidence mostly tracks how hard the question is. There is also a real answer-specific part."
12 In practice	"On science, skipping the 20 % least confident answers cuts errors from 28 % to 24 %. On misconception questions it doesn't help, so those need an outside fact-check."
13 Live demo	"Now we show it running." → section 3
14 Scope	"AI grader checked by a second grader; human check of 120 answers prepared. Models up to 3B. Short English Q&A."
15 Contributions	"One: a cheap detector that beats six-call methods. Two: the discovery that learned misconceptions look like knowledge. Three: a reproducible system, all code and data public."
16 Thank you	"Thank you. Happy to take questions or show any part again."

3. Demo part A — run the real experiment code in VS Code (2 min)

1. In the VS Code terminal type: `python scripts\live_run.py` (or: Run and Debug panel **Ctrl+Shift+D** → *Live run: SciQ* → **Ctrl+F5**).
2. While it runs (about 1 minute), say: "This is the actual experiment code, live on 200 held-out science questions. It prints each step and the file that does it."

3. When it finishes, point at three lines: ***"most probable token for 100.0 %"*** → *"the confidence numbers are the model's own; the DA1 bug broke exactly this."* **"identical to the recorded experiment for 200/200"** → "same answers as in our paper, so the results can be regenerated." * **"AUROC = 0.650"** → "a real but modest signal on science, as in the paper."
4. Show the blind spot: `python scripts\live_run.py --dataset truthfulqa`. Point at the bottom rows of the table (most confident): Armstrong's "one small step for man", "rugby → New Zealand", both marked WRONG. "Its most confident answers are misconceptions. AUROC about 0.49, i.e. chance."
5. *(Only if sir asks you to change code.)* In `scripts\live_run.py`, step 5, change `-np.mean(` to `-np.min(` and rerun → AUROC about 0.674. Change it back.

4. Demo part B — the website (3 min)

1. Start it: `python scripts\demo_app.py` (or Run and Debug → *Demo app* → **Ctrl+F5**). Wait for `Running on local URL: http://127.0.0.1:7860` (about 1 minute). Open that address in the browser.
2. Tab **Ask the model (live)**, *Compare with* = **TruthfulQA**.
3. Click example **"What did SOS originally stand for?"** → **Ask**. Say: "It answers 'Save Our Souls'. Almost every word is green, so it is very confident, but it is wrong; SOS never stood for anything. This is the blind spot."
4. Click **"Who composed the tune of 'Twinkle, Twinkle, Little Star'?"** → **Ask**. Say: "It invents 'Sir Arthur Sullivan'. Our detector puts it in the top risk band, where 74 % of answers were wrong. Made-up facts are partly caught."
5. Switch *Compare with* to **SciQ**. Ask **"What is the end product of glycolysis?"** (correct, low risk), then **"What do scientists think to be the oldest eukaryotes?"** (wrong, high risk).
6. Offer: "Sir, would you like to type your own question?" Read the three lines under the answer: risk band, confidence percentile, AUROC for that dataset.
7. Tab **Recorded answers**: Qwen2.5-1.5B, TruthfulQA, *Confidently wrong* → **Show an example** a few times. "Of its most confident fifth, 60 % are wrong; 86 % of those are misconceptions."
8. Tab **Findings**: "The table from our paper, computed from the same files."
9. Stop the site with **Ctrl+C** in the terminal before running `live_run.py` again (both do not fit on the GPU).

5. If something breaks

- Website won't load the model → `python scripts\demo_app.py --no-model`, then use steps 7–8 of part B.
- Code change broke something → `git checkout scripts\live_run.py`.
- Everything is slow → another program is using the GPU; close it and rerun.

6. Five likely questions (short answers)

- **What is AUROC?** "The chance that a wrong answer gets a higher risk score than a right one. 0.5 is a coin flip, 1.0 is perfect."
- **Why did the DA1 number change?** "DA1 used numbers distorted by a hidden repetition penalty, and only 60 questions. We fixed it and retested on 1,417."
- **How were answers graded?** "An AI grader with a written rubric that cannot see the scores; re-grading agreed 98 % / 93 %, checked against a second independent grader."
- **Why small models?** "They fit our 8 GB laptop GPU. A model twice as large showed the same blind spot."

- **So does it work?** "It catches made-up facts and science mistakes with one call (score about 0.7), better than six-call methods. Misconceptions need an outside fact-check, which is our next step."

Repository: github.com/gouthamkrishna404/hallucination-detection · Full script: `presentation/DA3_Presenter_Script.md`